



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۲ آذر

فصل چهارم: لایه *Network* بخش *Data – Plane* اسلاید ۴۱ تا ۶۱ [۱]

آدرس دهی *IP*

در گذشته این آدرس دهی *IP* به صورت *classful* بود و بخش آدرس، مضارب بایت بود. اگر مثلاً یک بایت اول برای زیر شبکه باشد، ۲ به توان ۲۴ تا *host* می‌توانستیم داشته باشیم. یا ۲ بیت، یا دو بیت یا ۳ بیت برای بخش زیر شبکه می‌توانستیم داشته باشیم. و اینگونه ممکن است اتلاف داشته باشیم.

CIDR = classless interdomain routing

در این روش، بخش زیر شبکه مضرب بایت نیست و می‌تواند مثلاً ۲۳ بیت باشد. که با همان ۲۳/ نشان می‌دهیم.

سوالات مطرح شده

سوال ۱: یک *host* چگونه آدرس خود را در یک سازمان بدست می‌آورد.

دو روش آن

روش اول: به صورت *hardcore* و دستی در فایل کانفیگ وارد کنیم.

روش دوم:

استفاده از پروتکل *DHCP = dynamic host configuration protocol*: در این روش آدرس *IP* به طور خودکار و داینامیک از یک سرور گرفته می‌شود. هر دفعه که به سرور وصل می‌شود ممکن است یک آدرس متفاوت بگیرد و فیکس نیست. این آدرس اختصاص یافته برای مدت محدودی در اختیار اوست. *host* می‌تواند آن را تمدید کند. اگر *host* ارتباط خود را قطع کند، آدرس آزاد می‌شود و دیگران می‌توانند از آن استفاده کنند. پروتکل *DHCP* یک پروتکل *client – server* است.

مراحل مختلف پروتکل DHCP

۱. *host* درخواست *DHCP discover* خود را پرودکست میکند.
۲. یکی از سرور های *DHCP* با ارسال پیام *DHCP offer* وجود خود را اعلام میکند و اعلام آمادگی میکند که میتواند به او آدرس *IP* دهد.
- این دو مرحله اول اختیاری بودند،
۳. بعد *Host* به اون *DHCP request* میدهد و درخواست آدرس *IP* میکند
۴. بعد سرور *DHCP* با ارسال *DHCP ACK* آدرس *IP* را به او میدهد.

سرور *DHCP* لازم نیست به صورت جداگانه طراحی شود و میتواند در روتر ها نیز آن ها را پیکر بندی کرد و هر *interface* یک زیر شبکه را مدیریت کند.

نکات پیاده سازی مربوط به کد در پروتکل DHCP

در مرحله *DHCP discover* اول تمام بیت های *src* (آدرس *IP* مبدا) برابر صفر و تمام بیت های *dest* (آدرس *IP* مقصد) برابر ۱ و به معنای پرودکست است. چون در ابتدا کاربر هیچ اطلاعی از شبکه ندارد. پورت مربوط به پروتکل *DHCP* برای کاربر برابر ۶۷ و برای سرور ۶۸ است. فیلد *yiaddr* در نهایت آدرسی است که به *client* داده می‌شود. اول صفر است. بعد یک سرور *DHCP* جواب میدهد که هست. پرودکست میکند. و فیلد *yiaddr* را هم پر میکند. برای آن یک *lifetime* هم مشخص میکند. در مرحله ۳ که مرحله *DHCP request* بود، باز کاربر، فیلد *src* خود را صفر میگذارد و پرودکست میکند اما اینبار *yiaddr* مساوی مقدار گرفته شده در مرحله ۲ است. بعد سرور با ارسال *DHCP ACK* با پرودکست کردن، و گذاشتن *yiaddr* اعلام میکند که اوکی است. آبدی تراکنش دو پیام اول با دو پیام دوم متفاوت است.

سوالات

۱. آدرس *this – network* یا این شبکه چیست؟
۲. چرا مشتری در گام ۳، به جای آدرس صفر، *yiaddr* را قرار نمیدهد.
۳. چرا در همه ۴ پیام ارسال شده، آدرس مقصد آدرس *broadcast* میباشد؟ استدلال کنید.

با اتصال به پروتکل *DHCP* علاوه بر آدرس *IP*، کاربر میتواند اطلاعات اضافی نیز بگیرد، مثلاً آدرس *IP* اولین روتر یا اسم و آدرس *DNS server* و پوشش شبکه. متوجه میشود که چه تعداد بیت از سمت چپ برای زیر شبکه است. پروتکل *DHCP* از *UDP* استفاده میکند. آن هم روی *IP* سوار است و آن هم روی *ETHERNET802.1*. در نهایت این فریم لایه دیتالینک است که روی شبکه میرود و برودکست میشود. آدرس برودکست با ۴۸ بیت ۱ نشان داده میشود.

روتر هم آن فریم را دریافت میکند و چون برودکست است میتواند آن را مطالعه کند. آدرس آیی مشتری، آدرس آیی اولین روتر در شبکه و نام و آدرس سرور *DNS* در پیام *DHCP ACK* میاید. در پروتکل استک مقصد، این فریم مرحله به مرحله دیماکس میشود.

سوالات

۲. یک شبکه چگونه آدرس زیر شبکه را بدست میآورد؟

هر *ISP* یک *ISP block* میگیرد که میتواند تا 2^{12} تا هاست را ساپورت کند. در واقع آدرس زیر شبکه آن ۲۰/ میخورد.

اما این بلوک را میتواند به ۸ تا بلوک کوچک تر تبدیل کند و به دپارتمان ها بدهد. (در آنجا، ۲۳/ میخورد) برای دسترسی از طریق اینترنت به آن دپارتمان ها از طریق *ISP* مربوطه شون میتوان به آنها وصل شد. اگر مثلاً یک دپارتمان از یک *ISP* به یک *ISP* دیگر برود، مدخل مربوط به *ISP* دوم تغییر نمیکند، و همچنین آدرس *IP* اون دپارتمان هم عوض نمیشود، فقط زیرشبکه ۲۳/ اون دپارتمان مربوطه را در مدخل *ISP* قرار میدهم. اگر از اینترنت کسی خواست با آن ارتباط بگیرد، چون با دو تا *ISP* اینجوری مطابق است، طبق *longest prefix match* به *ISP* دومی میرود.

ISP این بلوک را از *ICANN* میگیرد. اون این کار را از طریق ۵ تا *regional registry* انجام میدهد و هر کدام از آنها، این وظیفه را به *local registry* میدهند تا آنها تخصیص آدرس های *IP* را انجام دهند. برای حل مشکل کمبود *IP* از *NAT* استفاده میکنند. راه حل نهایی حرکت به سمت *IPv6* روی آوردند.

$$NAT = network\ address\ translation$$

با یک آدرس محلی *local network* میایند یک زیر شبکه درست میکند مثلاً با 2^8 تا هاست. بعد از طریق روتر، همه آنها را با یک آدرس *IP* مشخص به دنیا معرفی میکند. برای تمییز هاست ها در این شبکه محلی از شماره پورت استفاده میکنند.

همشون از دیدگاه اینترنت فقط یک آدرس *IP* دارند. پس لازم نیست که یک محدوده آدرس *IP* از *ISP* بگیرد.

این *ip* ها میتوانند تعویض شود ولی دنیای بیرون از آن آگاه نخواهد بود. و سازمان هم میتواند به راحتی *ISP* خود را عوض کند بدون تغییر ساختار داخلی خود مربوط به آدرس دهی های شبکه محلی شون.

پس چون این *IP* های محلی از بیرون قابل مشاهده نیست، یک لایه امنیتی ایجاد کرده.

در *NAT router* یک جدول نگاشت *NAT* وجود دارد. آدرس و پورت بسته ها را با آدرس خود روتر و شماره پورت جدید، جایگزین میکند. و آن را میفرستد. و موقع دریافت بسته های مربوط به هاست های محلی آن، باز از این جدول استفاده میکند.

پورت مربوط به *NAT router* چون ۱۶ بیت است، پس حداکثر حدود ۶۰ هزار تا تماس همزمان از شبکه خانگی میتواند رخ دهد.

مشکل های *NAT*:

۱. روتر باید هی تا لایه ۳ را بررسی کند.
۲. اینکه روتر، شماره پورت را تغییر دهد، عدول از بحث پایان پایان است چون روتر دیوایس لایه ۳ است ولی فیلد پورت برای لایه بالایی.

مشکل اصلی کمبود آدرس با *IPv6* حل می شود.

بعضی از کاربرد ها در شبکه های *NAT* دچار مشکل میشوند و پروتکل هایی برای رفع این مشکل آمده.

البته برای شبکه های سیار *4g* و *5g* کاربرد خوبی دارند.

سوال

چند مورد از این کاربرد ها را نام ببرید و دو پروتکل برای آن را بگویید.

IPv6

علاوه بر اینکه ۱۲۸ بیت است بجای ۳۲ بیت در *IPv4*، اندازه هدر آن ثابت و ۴۰ بایت است برعکس *IPv4* که متغیر بود.

علاوه بر این، از مفاهیم *flow* و *quality of service* هم پشتیبانی میکند که در *IPv4* نبود این چیزا.

مثلا فیلد *checksum* ندارد. تا پرودازش هدر سریعتر باشد.

فیلد آپشن حذف شده و به صورت مقداری که در *next header* قرار میگیرد، مشخص میشود. (زدم جمینای گفت مثلا به جای اینکه بده به *TCP* و به اون اشاره کنه و بگه بعدی هدر *TCP* است، میگه بعدی هدر روتینگ است)

اگر هم آپشن نباشد، فیلد *next header* میگوید که به کدام پروتکل لایه بالایی تحویل دهد. (بعدی هدر *TCP* است)

فیلد *priority* اجازه میدهد که با دیتاگرام های یک *flow* متفاوت و با تقدم متفاوت برخورد کنیم.

تعداد بیت های آدرس مبدا و مقصد هر کدام ۱۲۸ بیت هستند.
فیلد *flow label* نشان دهنده بسته های مرتب با یک *flow* یکسان را نشان میدهد. این فیلد توی همه اون ها یکسان میشود.
فیلد *hop limit* فکر کنم شبیه فیلد *TTL* است.
فیلد *payload len* هم طول پیلود را مشخص میکند.

مراجع

[1] “ch4-v8-slide-41-to-61,” <https://drive.google.com/drive/folders/>, 2022.